

MATLAB BASICS

What is MATLAB ?

- MATLAB is a proprietary software developed by Mathworks for numerical computing
- Available on all NCSU workstations
- Supported on Windows and Unix platforms
- Many toolboxes are available
- Easy to learn
- Programming done by writing scripts and functions
- Graphical libraries available

Assignment 0 – Matlab Academic Tutorial

- Go through first 2 sections of MATLAB on ramp tutorial videos at:
 - <https://matlabacademy.mathworks.com/>
- Please make sure you view at least the following 4 videos
 - [Working in The Development Environment](#) 5:21 min
 - [Getting Started with MATLAB](#) 7:00 min
 - [Writing a MATLAB Program](#) 4:57 min
 - [Publishing MATLAB Code from the Editor](#) 5:57 min
- Write a paragraph about what you learned most

Outline

- Basics
- Arrays
- Flow control
- Scripts and Functions
- Polynomials
- Plotting
- Symbolic Math

Basics

- Matlab commands can be entered interactively.
- Matlab m-files (program.m) can be created by the Matlab editor and then executed.
- Help window is available and should be used frequently.
- Default path is the work directory, all other paths where m-files reside should be set from file menu.
- By default all numbers are doubles except array indices.
- All variables and commands are case sensitive.

Basics

- Semi-colon “;” at the end of each line suppresses output.
- Comments are preceded by “%”
- Array indices cannot be zero or negative.
- MATLAB and its toolboxes have over 1000 canned routines for various computations
 - most have limitations on applicability
 - Multiple routines are available for the same computations
 - Most routines accept variable number of arguments

Basic Operations

- Multiply (*), Add (+), divide (/), power(^), transpose(')
- Element-wise operations for matrices and vectors
 - Multiply (.*), divide ./), power(.^)
- Logical operators
 - or (|), and (&)
 - less than (<), greater than (>)
 - less than or equal (<=), greater than or equal (>=)
 - equal (==), not equal (~=)
 - not (~)
- Line Continuation (...)

Arrays

- Numeric arrays
 - Vectors
 - Matrices
 - Multi-dimensional arrays
- Cell arrays
- Structure arrays

Vectors

- Row Vectors ($1 \times n$)
 - $a=[1, 2, 3];$
 - $a=[1 \ 2 \ 3];$
- Column Vectors ($n \times 1$)
 - $a=[1; 2; 3];$

Vector Initialization

- `a=zeros(10,1);` % column vector with all zeros
- `a=ones(10,1);` % column vector with all ones
- `a=zeros(1,10);` % row vector with all zeros
- `a=ones(1,10);` % row vector with all ones
- `a=0:2:10` % row vector
- `a=rand(10,1)` % column vector

Element-wise Vector Operations

- Operating ($\cdot$, $\cdot$, $\cdot$) every element of a vector with corresponding elements of another vector of the same size.
- Element-wise vector multiplication
 - $c = a \cdot b$
- Element-wise vector division
 - $c = a \cdot b$
- Element-wise vector exponentiation
 - $b = a \cdot 3$
 - $b = e \cdot a$

Other vector operations

- Size of a vector
 - `length(a)`
 - `size(a)`
- Vector Addition
 - $a=b+c;$
- Vector transpose (column to row or row to column)
 - $b=a'$.
- Vector multiplication (dot product)
 - $b=a*a'$

Matrices

- Matrices have rows and columns
- $A(5,4)$ – a matrix with 5 rows and 4 columns
- first index is row and second index is column
- Dimensions should match when performing matrix operations
- Matrix transpose is frequently used
 - $A=B'$
- Matrix size
 - $\text{Size}(A)$

Matrix Initialization

- `A=[1, 2, 3; 5, 6, 7; 8, 9, 10;]` % 3x3 matrix
- `A=[1 2 3; 5 6 7; 8 9 10;]` % 3x3 matrix
- `A=[1 2 3
3 4 5
6 7 8];` % 3x3 matrix
- `A=zeros(3,3)` or `A=zeros(3)` % 3x3 zero matrix
- `A=ones(3,3)` or `A=ones(3)` % 3x3 ones matrix
- `A=rand(3,3)` or `rand(3)` % 3x3 random matrix
- `A=eye(3)` % 3x3 identity matrix
- `A=diag(1:5)` % 3x3 diagonal matrix

Other matrix operations

- Referencing matrix elements
 - `B=A(:,3)` % extract 3rd column of matrix A
 - `B=A(2,:)` % extract 2nd row of matrix A
 - `B=A(1,1:3)` % extract first row, column 1 to 3 of A
- Matrix multiplication
 - `A=B*C` % $A(m \times m)$ $B(m \times n)$ $C(n \times m)$
 - `A=B*C'` % $A(m \times m)$ $B(m \times n)$ $C(m \times n)$
- Matrix division (right division)
 - `A=B/C` % $A, B, C(n \times n)$, $A=B*\text{inv}(C)$
- Matrix solution (left division)
 - `x=A\b` % $A(n \times n)$, $b(n \times 1)$, $x(n \times 1)$, $x=\text{inv}(A)*b$

Element-wise Matrix Operations

- Operating ($\cdot$, $\cdot$, $\cdot$) every element of a matrix with corresponding elements of another matrix of the same size.
- Element-wise matrix multiplication
 - $A=A.*B$
- Element-wise matrix division
 - $A=A./B$
- Element-wise matrix exponentiation
 - $A=A.^3$
 - $A=e.^A$

Multi-dimensional Arrays

- Arrays that have more than 2 dimensions
- `A=rand(5,3,2)` % 5 x 3 x 2 array
- Basic operations are similar to matrices
- Transposes not available

Exercises with Matrices, Vectors and Arrays

- Construct vectors of length 10 using the 6 methods outlined earlier
- Find element-wise multiplication and dot-products of two of these vectors
- Perform element-wise division, power on these vectors.
- Repeat the same for matrices (use 5x5 and 5x2 matrices).
 - Display $A(:,3)$, $A(1,:)$ and perform vector operations
- Generate a random array A of size 5x3x2.
 - Display $A(5,3,:)$, $A(:,3,2)$

Array Functions

`[u,v,w] =
find(A)`

Computes the arrays u and v , containing the row and column indices of the nonzero elements of the matrix A , and the array w , containing the values of the nonzero elements. The array w may be omitted.

`length(A)`

Computes either the number of elements of A if A is a vector or the largest value of m or n if A is an $m \times n$ matrix.

Array Functions

`max (A)`

Returns the algebraically largest element in A if A is a vector.

Returns a row vector containing the largest elements in each column if A is a matrix.

If any of the elements are complex, `max (A)` returns the elements that have the largest magnitudes.

Array Functions

```
[x,k] = max(A)
```

Similar to `max(A)` but stores the maximum values in the row vector `x` and their indices in the row vector `k`.

```
min(A)
```

and

```
[x,k] = min(A)
```

Like `max` but returns minimum values.

Array Functions

`size(A)`

Returns a row vector $[m \ n]$ containing the sizes of the $m \times n$ array A .

`sort(A)`

Sorts each column of the array A in ascending order and returns an array the same size as A .

`sum(A)`

Sums the elements in each column of the array A and returns a row vector containing the sums.

More Array Exercises

- Solve the following problems in **Palm** Text
 - 2.19 – cost estimation example (p. 100)
 - 2.24 – tensile force calculation example (p. 102)

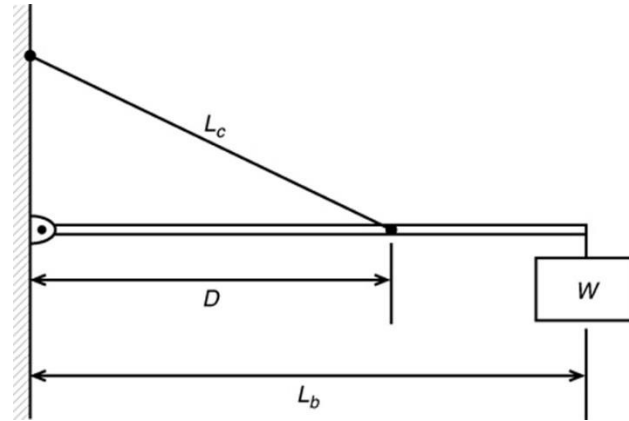
Cost Estimation Example (Palm 2.19, p. 100)

- Create 5 x 3 matrix for quantity purchased
- Create 5 x 1 vector for price per ton
- What is the total spent in May, June, July?
- What is the total spent for each material?
- What is the total spent on all materials?

Material	Price (\$/ton)	Quantity Purchased (tons)		
		May	June	July
1	300	5	4	6
2	550	3	2	4
3	400	6	5	3
4	250	3	5	4
5	500	2	4	3

Tensile Force Calculation Example (Palm 2.24, p. 102)

- Tension T in Cable
- $W = 400 \text{ N}$, $L_b = 5 \text{ m}$, $L_c = 5 \text{ m}$.
- Find D s.t T is minimum using element-wise array operations and the MATLAB `min` function
- Find the values of D that result in T within 10% of the minimum value using the MATLAB `find` function



$$T = \frac{L_b L_c W}{D \sqrt{L_c^2 - D^2}}$$

Cell arrays

- Each element of a cell array can be an array of different type and size
- Can be used represent databases
- Example: environmental database
 - Location
 - Date
 - Air temperatures at 8 am, 12 noon, 5 pm
 - Water temperature at three locations at 8 am, 12 noon, 5 pm

Cell Array example: Environmental Database

- Cell indexing:
 - $A(1,1) = \{\text{'Neuse River Basin'}\};$
 - $A(1,2) = \{\text{'Jan 10, 2005'}\};$
 - $A(2,1) = \{[50, 60, 58]\};$
 - $A(2,2) = \{[45,47,46; 44,46,45; 42,45,43]\};$
- Content indexing:
 - $A\{1,1\} = \text{'Neuse River Basin'};$
 - $A\{1,2\} = \text{'Jan 10, 2005'};$
 - $A\{2,1\} = [50, 60, 58];$
 - $A\{2,2\} = [45,47,46; 44,46,45; 42,45,43];$

Neuse River Basin	Jan 10, 2005
[50 60 58]	$\begin{bmatrix} 45 & 47 & 46 \\ 44 & 46 & 45 \\ 42 & 45 & 43 \end{bmatrix}$

Cell Array Functions

Function	Description
<code>C = cell(n)</code>	Creates an $n \times n$ cell array <code>C</code> of empty matrices.
<code>C = cell(n,m)</code>	Creates an $n \times m$ cell array <code>C</code> of empty matrices.
<code>celldisp(C)</code>	Displays the contents of cell array <code>C</code> .
<code>cellplot(C)</code>	Displays a graphical representation of the cell array <code>C</code> .
<code>C = num2cell(A)</code>	Converts a numeric array <code>A</code> into a cell array <code>C</code> .
<code>[X,Y, ...] = deal(A,B, ...)</code>	Matches up the input and output lists. Equivalent to $X = A, Y = B, \dots$
<code>[X,Y, ...] = deal(A)</code>	Matches up the input and output lists. Equivalent to $X = A, Y = A, \dots$
<code>iscell(C)</code>	Returns a 1 if <code>C</code> is a cell array; otherwise, returns a 0.

Cell Array Exercise 1

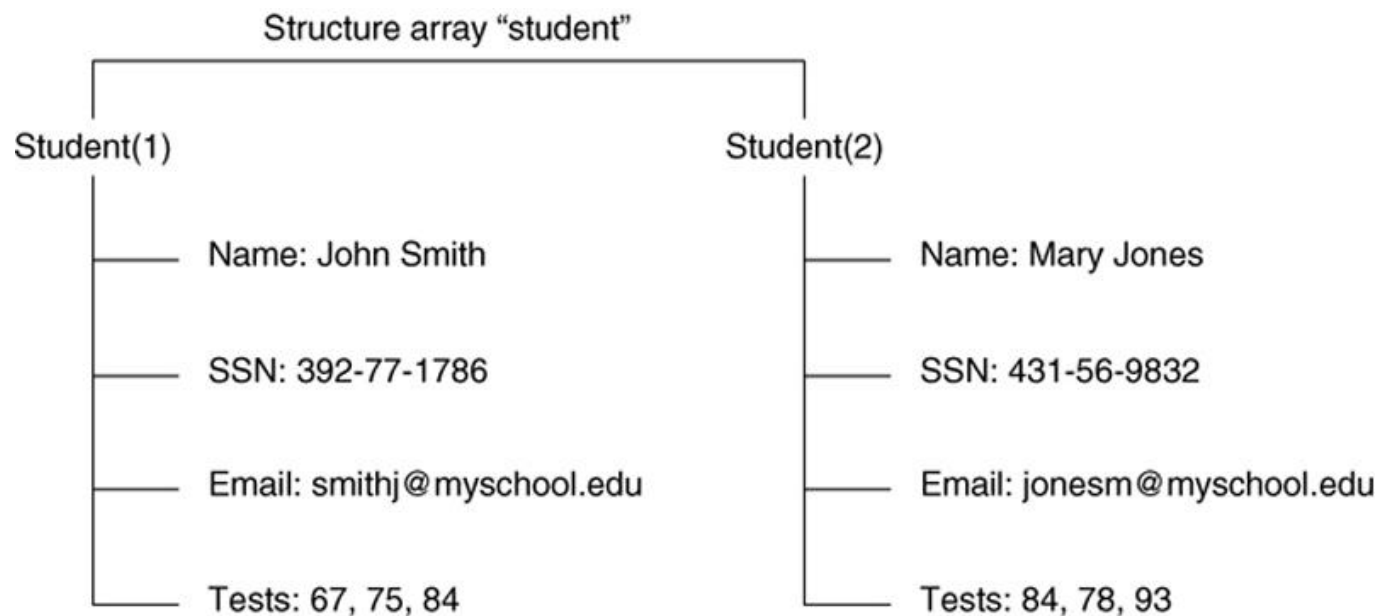
- Palm 2.61 (p. 119)

Cell Array Exercise 2

- Palm 2.62 (p. 119)

Structure arrays

- Structure arrays can be used to store dissimilar objects in a single array (text string, numbers etc.)
- E.g.,



Structure Arrays Example

```
student(1).name = 'John Smith';  
student(1).SSN = '392-77-1786';  
student(1).email = 'smithj@ncsu.edu';  
student(1).tests = [67,75,84];  
student(2).name = 'Mary Jones';  
student(2).SSN = '431-56-9832';  
student(2).email = 'jonesm@ncsu.edu';  
student(2).tests = [84,78,93];
```

Structure Functions

Function

```
names =  
fieldnames(S)
```

```
F =  
getfield(S, 'field')
```

```
isfield(S, 'field')
```

Description

Returns the field names associated with the structure array S as names, a cell array of strings.

Returns the contents of the field 'field' in the structure array S. Equivalent to F = S.field.

Returns 1 if 'field' is the name of a field in the structure array S, and 0 otherwise.

Structure Functions

```
S = rmfield(S,'field')
```

Removes the field 'field' from the structure array S.

```
S = setfield(S,'field',V)
```

Sets the contents of the field 'field' to the value V in the structure array S.

```
S =  
    struct('f1','v1','f2','v2'  
          ',...)
```

Creates a structure array with the fields 'f1', 'f2', ... having the values 'v1', 'v2',

Structure Array Exercise

- Palm 2.64 (p. 119) – Bridge example

Array Examples -

array_examples.m

- This example contains various vector, matrix, and array operations.
- Study this example.
- Run this example.
- Modify to perform other array operations
 - sum, prod, norm, max, min, cat, find

Scripts and Functions

- Script M-files are very convenient (main program).
- Functions can be saved in separate m-files and called from scripts.
- Functions can have sub functions in the same m-file.
- Scripts can also be named as functions that return nothing (dummy function)
 - This makes it convenient to keep all functions in the same script.
- Function names can be passed as an argument to other functions.

Scripts and Functions (contd.)

```
function main_function
% do stuff
...
% call sub function 1
[output1] = subfunction1(input1,input2);
...
% call subfunction 2
[output2,output3] = subfunction2(input3);
...
end
```

```
function [output1] = subfunction1(input1,input2)
% do stuff
...
end
```

```
function [output2, output3] = subfunction2(input3)
% do stuff
...
end
```

Function Example – test_script.m

```
% main function

function test_script
    x=rand(100,1); % x is a column vector
    [mean, stdev]=stat(x);
    fprintf('mean = %g\n',mean);
    fprintf('stdev = %g\n',stdev);

end

function [mean,stdev] = stat(x)
    %STAT Interesting statistics.
    n = length(x);
    mean = avg(x,n);
    stdev = sqrt(sum((x-avg(x,n)).^2)/n);

end

function mean = avg(x,n)
    %MEAN subfunction
    mean = sum(x)/n;

end
```

Functions (contd)

- Functions can be invoked in 4 ways
 - As a character string
 - As a function handle (preferred)
 - As an inline function object
 - As a string expression

Function Handles

You can create a function handle to any function by using the *at* sign, @, before the function name. You can then name the handle if you wish, and use the handle to reference the function. For example, to create a handle to the sine function, you type

```
>>sine_handle = @sin;
```

where `sine_handle` is a user-selected name for the handle.

(continued ...)

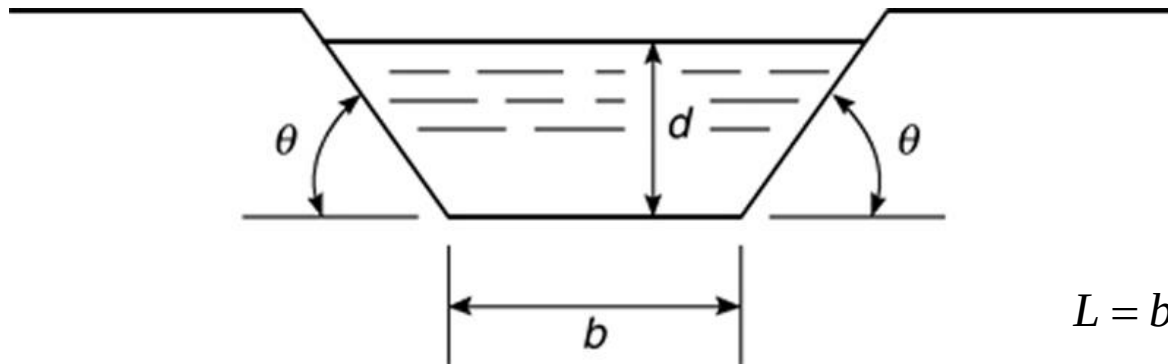
Function Handles

A common use of a function handle is to pass the function as an argument to another function. For example, we can plot $\sin x$ over $0 \leq x \leq 6$ as follows:

```
>>x=-pi:pi/10:pi  
>>plot(x, sine_handle(x))
```

Irrigation Example (Palm p. 137)

- Find d , b , θ such that the wetted perimeter (L) is minimum and cross sectional area (A) is 100 ft^2



$$L = b + \frac{2d}{\sin \theta}$$

$$b = \frac{A}{d} - \frac{d}{\tan \theta}$$

- Use `fminsearch` function

Irrigation Example (Contd.)

Relational Operators

Operator	Meaning
<	Less than.
<=	Less than or equal to.
>	Greater than.
>=	Greater than or equal to.
==	Equal to.
~=	Not equal to.

Logical Operators

Operator	Name	Definition
~	NOT	$\sim A$ returns an array the same dimension as A ; the new array has ones where A is zero and zeros where A is nonzero.
&	AND	$A \ \& \ B$ returns an array the same dimension as A and B ; the new array has ones where both A and B have nonzero elements and zeros where either A or B is zero.
	OR	$A \ \ B$ returns an array the same dimension as A and B ; the new array has ones where at least one element in A or B is nonzero and zeros where A and B are both zero.
&&	Short-Circuit AND	Operator for scalar logical expressions. $A \ \&\& \ B$ returns true if both A and B evaluate to true, and false if they do not.
	Short-Circuit OR	Operator for scalar logical expressions. $A \ \ B$ returns true if either A or B or both evaluate to true, and false if they do not.

Order of precedence for operator types

Precedence	Operator type
First	Parentheses; evaluated starting with the innermost pair.
Second	Arithmetic operators and logical NOT (~); evaluated from left to right.
Third	Relational operators; evaluated from left to right.
Fourth	Logical AND.
Fifth	Logical OR.

Logical functions

Logical function	Definition
<code>all(x)</code>	Returns a scalar, which is 1 if all the elements in the vector x are nonzero and 0 otherwise.
<code>all(A)</code>	Returns a row vector having the same number of columns as the matrix A and containing ones and zeros, depending on whether or not the corresponding column of A has all nonzero elements.
<code>any(x)</code>	Returns a scalar, which is 1 if any of the elements in the vector x is nonzero and 0 otherwise.
<code>any(A)</code>	Returns a row vector having the same number of columns as A and containing ones and zeros, depending on whether or not the corresponding column of the matrix A contains any nonzero elements.
<code>finite(A)</code>	Returns an array of the same dimension as A with ones where the elements of A are finite and zeros elsewhere.

Logical Functions

Logical function	Definition
ischar(A)	Returns a 1 if A is a character array and 0 otherwise.
isempty(A)	Returns a 1 if A is an empty matrix and 0 otherwise.
isinf(A)	Returns an array of the same dimension as A, with ones where A has 'inf' and zeros elsewhere.
isnan(A)	Returns an array of the same dimension as A with ones where A has 'NaN' and zeros elsewhere. (‘NaN’ stands for “not a number,” which means an undefined result.)

Logical Functions

`isnumeric(A)`

Returns a 1 if A is a numeric array and 0 otherwise.

`isreal(A)`

Returns a 1 if A has no elements with imaginary parts and 0 otherwise.

`logical(A)`

Converts the elements of the array A into logical values.

`xor(A,B)`

Returns an array the same dimension as A and B; the new array has ones where either A or B is nonzero, but not both, and zeros where A and B are either both nonzero or both zero.

Flow Control

- for
- while
- if
- switch
- break
- continue
- end

for loops

```
for loop variable = start:increment:end  
    statements;  
end
```

```
for k = 5:10:35  
    x = k^2;  
end
```

```
for k = 1:n  
    for j = 1:n  
        A(k,j) = 1/(k+j-1);  
    end  
end
```

While Loops

```
while (expr)
    statements
end
```

```
x=5;
while x < 25,
    disp(x);
    x = 2*x-1;
end
```

if – end

```
if (expr)  
  statements
```

```
end
```

```
if (x >= 0)  
  y = sqrt(x);  
end
```

if-else– end

```
    if (expr)  
        statements1  
    else  
        statements2  
    end
```

```
if (x >= 0)  
    y = sqrt(x) ;  
else  
    y = exp(x) - 1 ;  
end
```

if-elseif-else– end

```
if (expr1)  
    statements1  
elseif (expr2)  
    statements2  
else  
    statements3  
end
```

```
if (x > 10)  
    y = log(x);  
elseif (x >= 0)  
    y = sqrt(x);  
else  
    y = exp(x)-1;  
end
```

switch

```
switch (expr)  
    case value1  
        statements1  
    case value2  
        statements2  
    otherwise  
        statements3  
end
```

switch

```
method = input('Type linear, bilinear, or cubic.\n','s');
method = lower(method);
switch method
    case {'linear','bilinear'}
        disp('Method is linear');
    case 'cubic'
        disp('Method is cubic');
    otherwise
        disp('Unknown method.');
```

end

break, continue

- `break` exits out of a `for` or `while` loop.
- `continue` passes control to the next iteration of the `for` or `while` loop

Flow control example – [flow control examples.m](#)

- Contains examples of for, if, while, and switch statements.
- Study and run this example.
- Modify and run again.

Symbolic Math

- Creating and manipulating symbolic variables
 - `sym`, `syms`, `sym2pol`, `pol2sym`
- Algebraic simplification
 - `simplify`, `factor`, `expand`, `collect`, `simple`, `pretty`
- Taylor series expansions
 - `taylor`
- Integration
 - `int`
- Differentiation
 - `diff`
- Other sym computations
 - `svd`, `eig`, `dsolve`

Symbolic Math Examples – [symbolic math examples.m](#)

- Contains various symbolic math operations such as
 - Simplification, factorization, Integration, differentiation, taylor series expansion etc.
- Study and run this example.

Polynomials

- representation
- polyval
- roots
- conv
- deconv
- polyadd

Polynomial Representation

- Example

$$p(x) = x^3 - 2x - 5$$

- Representation
 - `p=[1 0 -2 -5];`
- Symbolic representation
 - `poly2sym(p)`

Polynomial roots

- Use roots function to evaluate all roots of a polynomial p
 - `x=roots(p)`
- Roots can also be found using fzero function
 - `x=fzero(fun_p,x0)`

Polynomial Evaluation

- Use function `polyval` to evaluate a polynomial
 - `a=polyval(p,x)`

Other Poly functions

- Construct polynomial with roots x
 - `p = poly(x)`
- Multiply polynomials
 - `r = conv(p,q)` % r is a vector of length(p)+length(q)-1
- Divide polynomials (q/p)
 - `[u,v] = deconv(q,p)` % u is result and v is remainder
- Evaluate derivative of polynomials
 - `q = polyder(p)`

Polynomial Example – poly_examples.m

- Contains examples showing
 - how to construct, solve, multiply, and divide polynomials
 - How to use symbolic representation
- Study and run this example
- Modify and run again

MATLAB Plotting

- Various types of 2D plots
 - ezplot, plot, subplot, ezpolar, polar, plotyy, loglog, semilogx, semilogy, bar, barh, stairs, hist, pie
- Cylinder, polar, sphere etc.
 - ezpolar, polar, sphere, cylinder, compass, rose, coneplot
- Surface plots
 - ezsurf, surf, surfc, surface
- Contour plots
 - ezcontour, contour, contourc, contourf, contour3, contourslice

Plotting

- 3D plots
 - Plot3, bar3, bar3h
- Transformation
 - meshgrid, cart2pol, sph2cart, cart2sph, surf2patch, trimesh, delauny, isosurface, smooth, smooth3
- Animation
 - getframe, movie
- Sounds
 - sound
- Appearance
 - pcolor, colormap, axis, shading, patch, set

ezplot

- ezplot is an easy to use *function* plotter
- If you want to know how a function looks like use ezplot
 - `ezplot('sin(x^2)')`
- Default interval is $[-2\pi, 2\pi]$
- Interval can be changed using optional argument
 - `ezplot('cos(x^3)', [0, pi])`

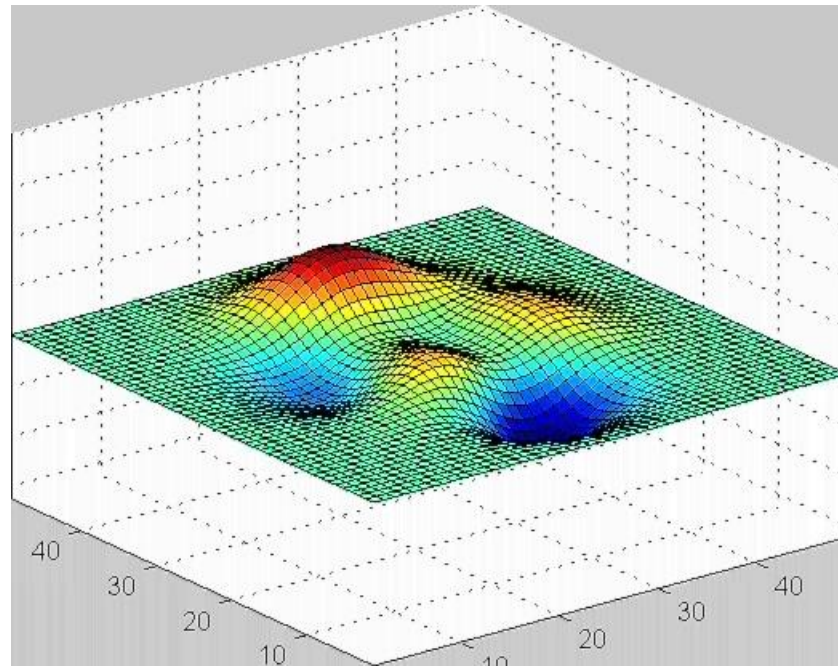
plot

- plot is used to plot xy line plots
 - `plot(x1,y1,'-go',x2,y2,'-r+')`
- “hold on” can be used to add lines to a current plot. Following three lines are equivalent to the previous line
 - `plot(x1,y1,'-go');`
 - `hold on;`
 - `plot(x2,y2,'-r+');`

Plotting – plot_examples.m

- This script contains various plotting examples
- study, and run this script
- Modify to change plotting appearances etc.

Animation example



Input/Output

- Matlab provides many functions for reading and writing to files, reading from command line, writing to screen etc.

Input/output

- fopen/fclose – open/close a file
- fread/fwrite – read/write binary data from/to a file
- fscanf/fprintf – read/write ascii (text) data from/to a file
- sscanf/sprintf – read/write from/to a string
- save/load – write/read matlab variables
- input – read keyboard input
- fgetl, fgets – read line
- xlsread/xlswrite – import/export data from/to MS Excel spreadsheet
- disp, error – display to screen
- Create an m-file of data and call it like a function

Input/Output – io_examples.m

- Contains Input/Output examples for
 - fopen, fread, fwrite, fclose
 - fprintf, fscanf
 - save, load
 - disp, sprintf, sscanf
- study and run this code

Debugging in Matlab

- Breakpoints
 - collatz example
- M-Lint code analyzer
- Cells

Publish feature in MATLAB

- Publishing code and output to msword, pdf, html files
- Use “%%” to divide code into sections
- You can use latex style commands to typeset equations
- Preferred way to submit assignments in this class

Simple MATLAB example problems

- Parachute example
- Truss example
- Reactor example

Parachute Example (C&C p. 13)

- Newton's second law ($f = ma$) can be applied to the falling parachutist problem.
- Two forces act, upward drag force f_u , and downward gravity force f_d

$$\downarrow f_d - f_u = ma$$

$$mg - cv = m \frac{dv}{dt}$$

$$\frac{dv}{dt} = g - \frac{c}{m}v$$

RHS of ODE



Falling Parachutist Problem (contd.)

- ode can be solved analytically for the velocity $v(t)$ by separating the variables and then simply integrating.

$$\int \frac{dv}{g - \frac{c}{m}v} = \int dt$$

$$-\frac{m}{c} \ln \left(g - \frac{c}{m}v \right) = t + \text{const}$$

Falling Parachutist Problem (contd.)

- If the initial condition is $v = 0$ at $t = 0$,

$$\text{const} = -\frac{m}{c} \ln(g)$$

- Substituting we obtain

$$\begin{aligned} t &= -\frac{m}{c} \ln\left(g - \frac{c}{m} v\right) + \frac{m}{c} \ln(g) \\ &= -\frac{m}{c} \ln\left(\frac{g - \frac{c}{m} v}{g}\right) = -\frac{m}{c} \ln\left(1 - \frac{cv}{mg}\right) \end{aligned}$$

Falling Parachutist Problem (contd.)

- Taking exponential on both sides,

$$e^{-\frac{ct}{m}} = 1 - \frac{cv}{gm}$$

- Therefore,

$$v(t) = \frac{gm}{c} \left(1 - e^{-\frac{ct}{m}} \right)$$

Numerical Solution

- Numerical solution can be obtained for this problem using forward finite-differencing (forward Euler).

$$\frac{dv}{dt} = g - \frac{c}{m}v$$
$$\frac{v(t + \Delta t) - v(t)}{\Delta t} = g - \frac{c}{m}v(t)$$

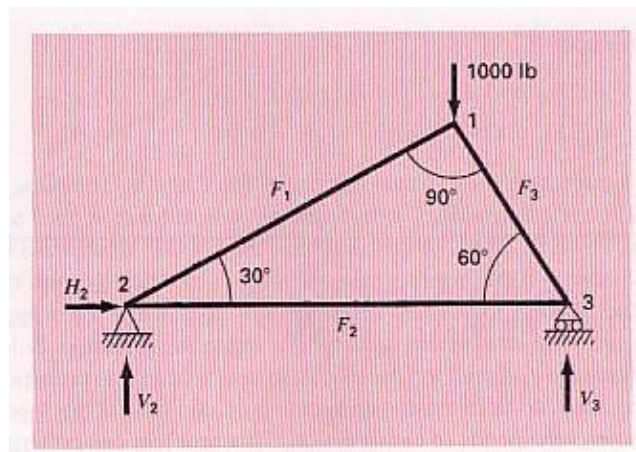
$$v(t + \Delta t) = v(t) + \Delta t \left(g - \frac{c}{m}v(t) \right)$$

MATLAB implementation – In class exercise

- Write MATLAB code to solve the parachute problem using analytical and numerical (forward Euler) methods and compare.
- Assume the following values:
 - $g = 9.8 \text{ m/s}^2$, $m = 68.1 \text{ kg}$, $c = 12.5 \text{ kg/s}$
 - initial condition $t=0$, $v=0$
- Solve for $t = 0$ to 12 secs.
 - What is the desired numerical time step size ?
- Solve again for $t = 0$ to 50 secs
 - What is a reasonable estimate of terminal velocity?

Truss Example (C&C p. 318)

- Statically determinate truss
- Simply balance horizontal and vertical forces at each node
 - Compressive is positive
- 6 linear equations 6 unknowns

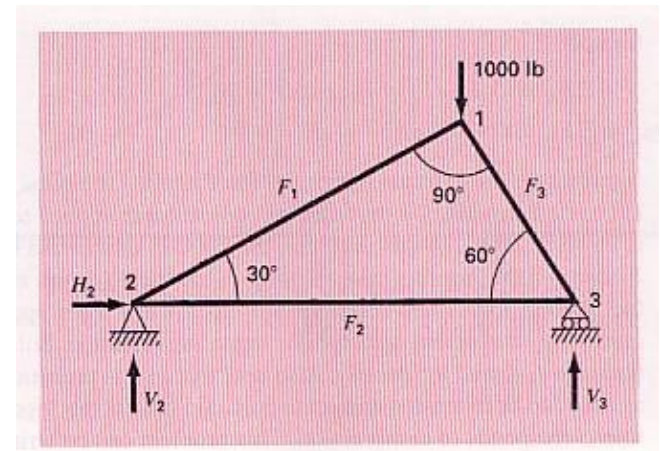


Truss Example (contd.)

$$\text{node 1} \begin{cases} \sum F_H = -F_1 \cos 30 + F_3 \cos 60 = 0 \\ \sum F_V = -F_1 \sin 30 - F_3 \sin 60 - 1000 = 0 \end{cases}$$

$$\text{node 2} \begin{cases} \sum F_H = F_2 + F_1 \cos 30 + H_2 = 0 \\ \sum F_V = F_1 \sin 30 + V_2 = 0 \end{cases}$$

$$\text{node 3} \begin{cases} \sum F_H = -F_2 + F_3 \cos 60 = 0 \\ \sum F_V = F_3 \sin 60 + V_3 = 0 \end{cases}$$

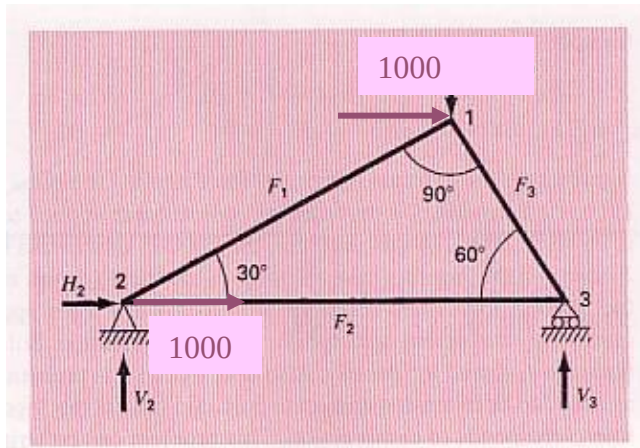


Truss Example (contd.)

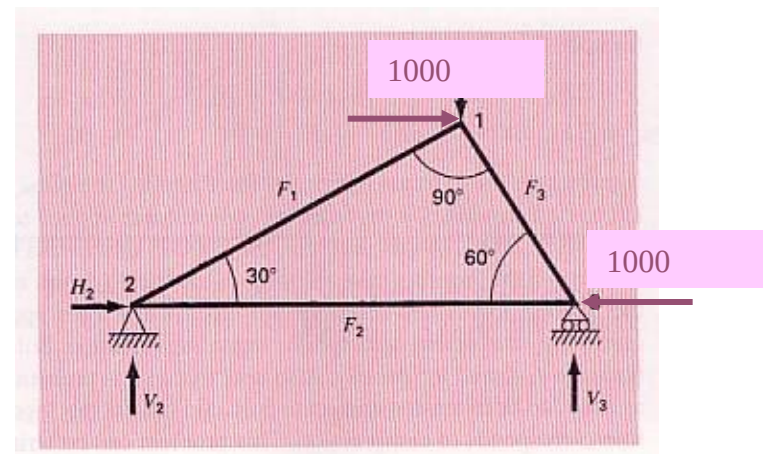
$$\begin{bmatrix} -0.866 & 0 & 0.5 & 0 & 0 & 0 \\ -0.5 & 0 & -0.866 & 0 & 0 & 0 \\ 0.866 & 1 & 0 & 1 & 0 & 0 \\ 0.5 & 0 & 0 & 0 & 1 & 0 \\ 0 & -1 & -0.5 & 0 & 0 & 0 \\ 0 & 0 & 0.866 & 0 & 0 & 1 \end{bmatrix} \begin{Bmatrix} F_1 \\ F_2 \\ F_3 \\ H_2 \\ V_2 \\ V_3 \end{Bmatrix} = \begin{Bmatrix} 0 \\ 1000 \\ 0 \\ 0 \\ 0 \\ 0 \end{Bmatrix}$$

Truss Example – MATLAB implementation

- Write a matlab code to solve this problem for the 6 unknown forces
- How would you solve this problem if the external forces are changed at each node?



Wind from left



Wind from right

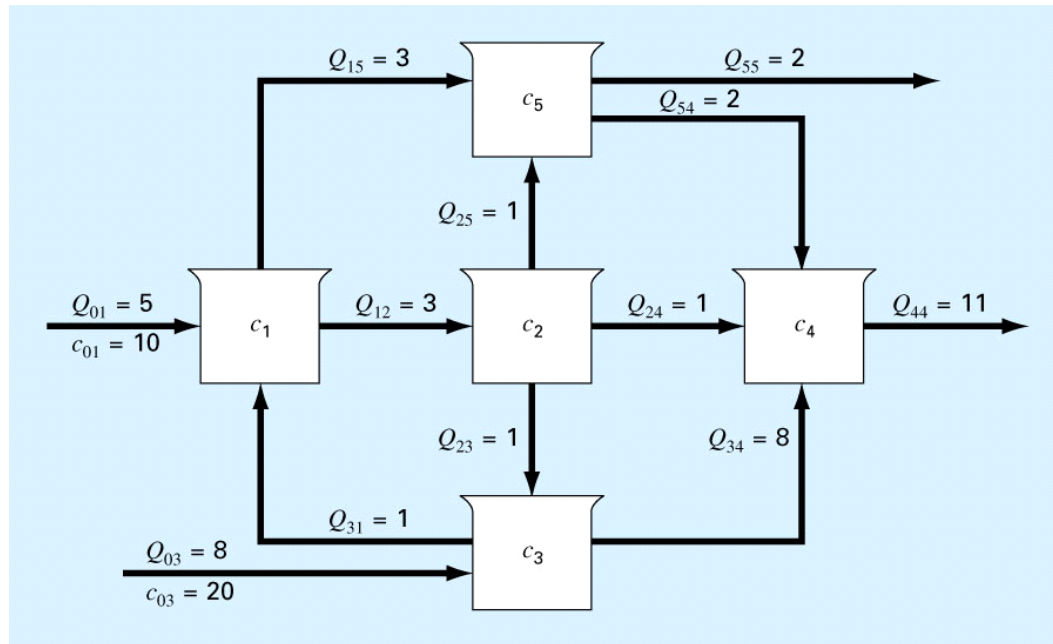
Reactor Example (C&C 305)

- Five completely mixed chemical reactors linked by pipes
- Volume flow rates and inlet concentrations are known
- Concentrations in each reactor are unknown
- Five mass balance equations five unknowns
- Similar to truss problem

Reactor Example

- For reactor i ,

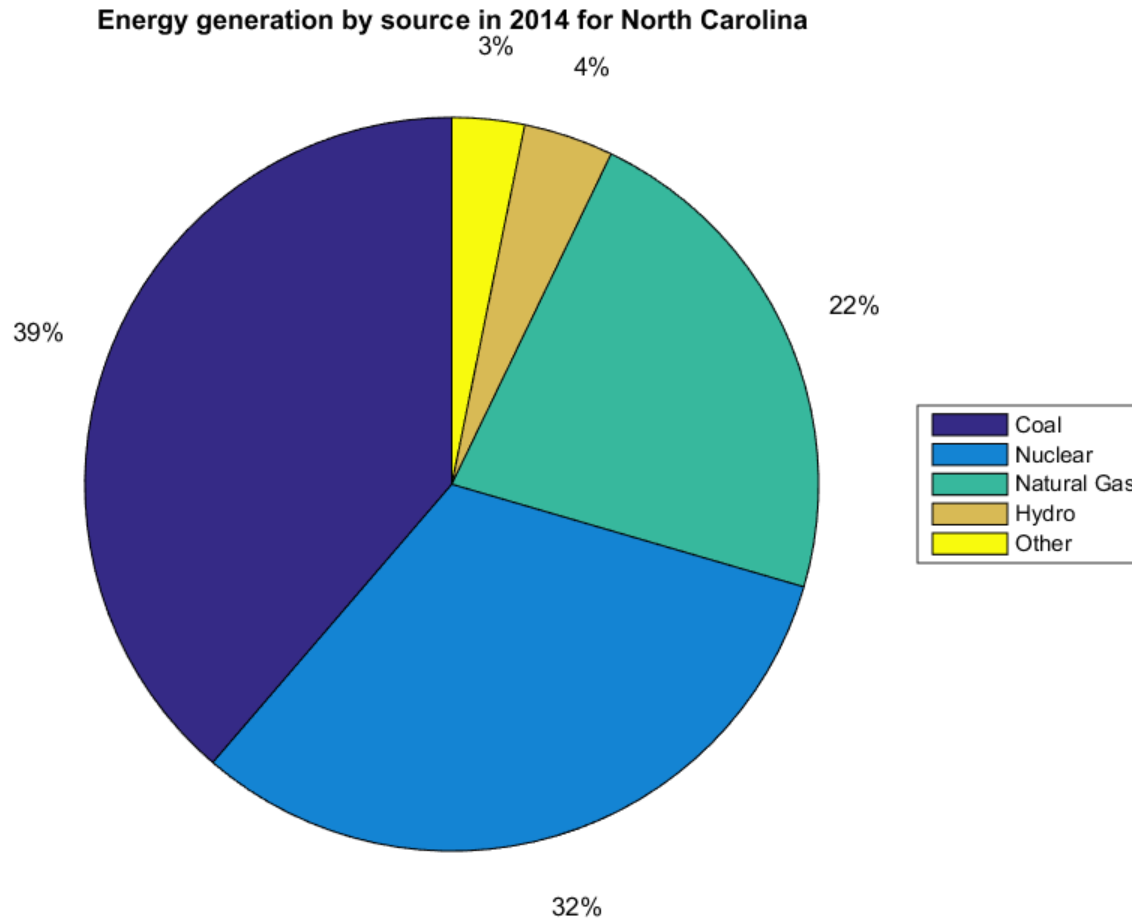
$$\sum Q_{i,in} C_{i,in} = \sum Q_{i,out} C_i$$



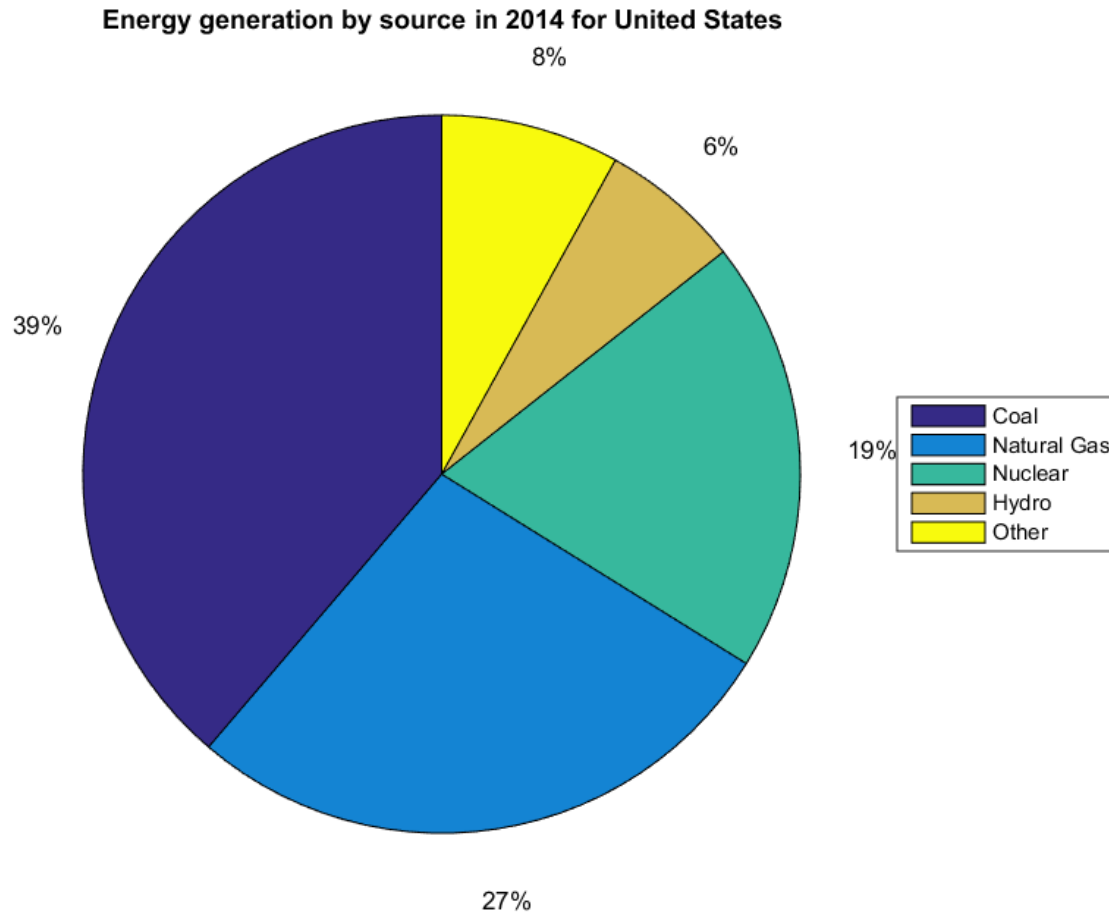
Plot energy data from excel file

- Extract needed data from excel file and plot pie chart for a given state
- Main functions used:
 - input
 - xlsread
 - strcmp
 - pie

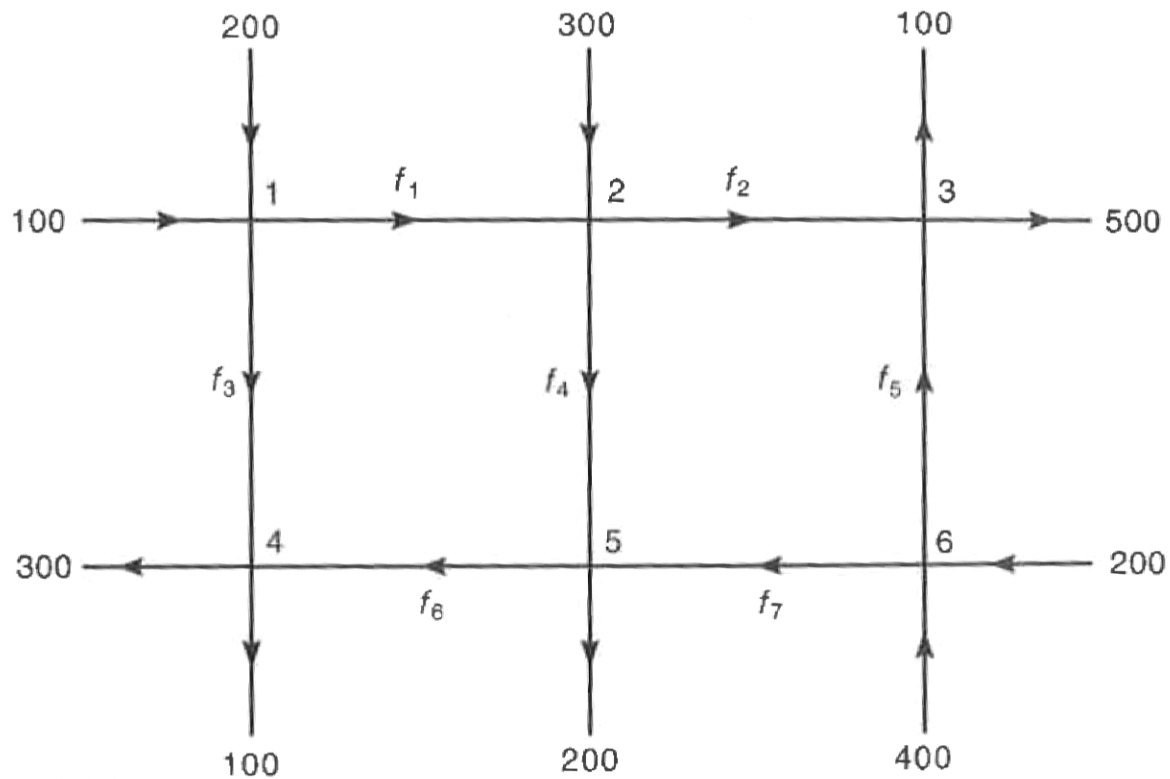
Plot energy data for NC



Plot energy for US



Traffic Example (Palm 2.47)



Traffic Example

$$A = [\dots];$$

$$f_1 = (\dots) + () f_6 + () f_7$$

$$\textcircled{1} f_1 + f_3 = 300$$

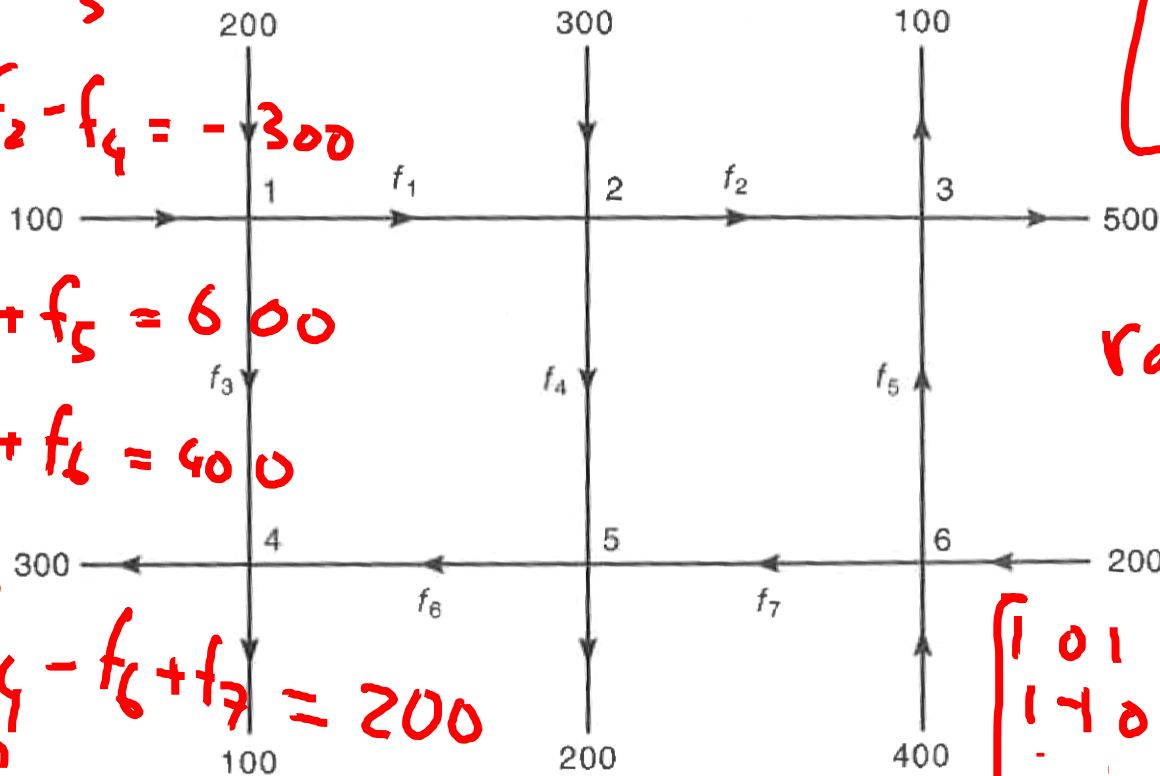
$$\textcircled{2} f_1 - f_2 - f_4 = -300$$

$$\textcircled{3} f_2 + f_5 = 600$$

$$\textcircled{4} f_3 + f_6 = 400$$

$$\textcircled{5} f_4 - f_6 + f_7 = 200$$

$$\textcircled{6} f_5 + f_7 = 600$$



$$A = \begin{bmatrix} 1 & 1 & 1 \\ 2 & 2 & 2 \\ 3 & 3 & 3 \end{bmatrix}$$

$\text{rank}(A)$

$$\begin{bmatrix} 1 & 0 & 1 & 300 & 0 & 0 \\ 1 & -1 & 0 & -1 & 0 & 0 \\ \vdots & \vdots & \vdots & \vdots & \vdots & \vdots \\ 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix}_{7 \times 7}$$

'rank'

$$AA = \begin{bmatrix} 1 & 0 & 0 & 0 & 0 & -1 & 0 & -100 \\ \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots \end{bmatrix}$$

$f_1 = f_6 - 100$

$$AA = [A \quad b]$$

$$\text{ref}(AA)$$

$$f_1 - f_6 - f_7 = b_1$$

$$f_1 = b_1 + f_6 + f_7$$

$$= \begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 & (b_1) \\ 0 & 1 & 0 & 0 & 0 & 0 & (b_2) \\ \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots \\ f_5 & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots \end{bmatrix}$$

$\begin{matrix} 0 & -100 \\ -1 & \nearrow \end{matrix}$

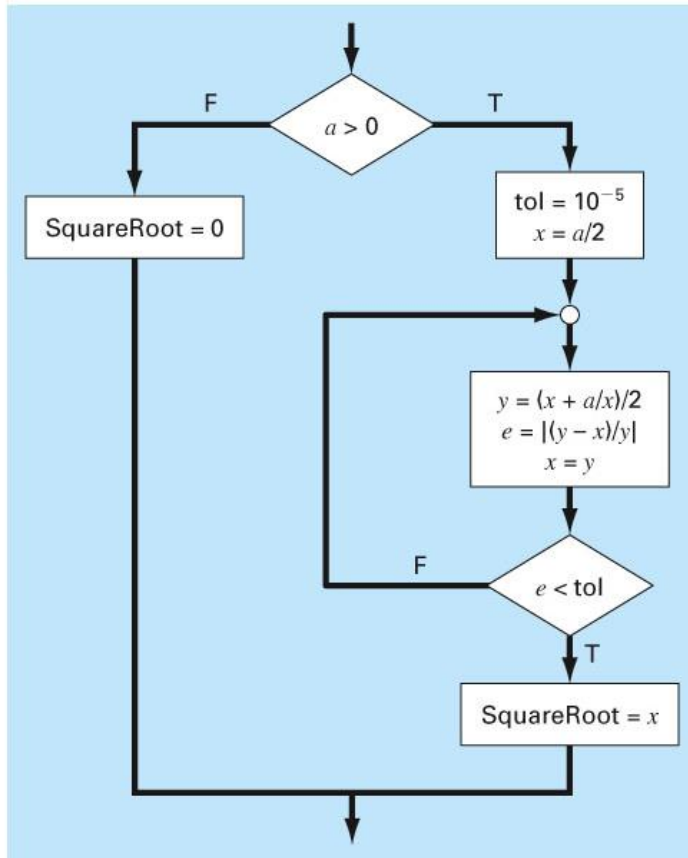
Calculating Square Root (C & C 2.7)

- To calculate the square root of a , iteratively calculate x starting from a value of $a/2$

$$x = \frac{x + a / x}{2}$$

- At the end of sufficient number of iterations (i.e., convergence), x will converge to $\text{sqrt}(a)$

From Algorithm to Code - Square Root



Algorithm Flow Chart

```
function [x] = square_root(a)
% return 0 for non-positive values of a
if (a <= 0)
    x = 0;
    return
end
% set initial values
tol=1e-6; % tolerance
e = inf; % set e to a large number
x = a/2; % initialize x
while (e > tol)
    y = (x+a/x)/2;
    e = abs(y-x)/y;
    x = y;
end
end
```

Code